

STORAGE MANAGEMENT (FILE SYSTEMS)

ICT1308

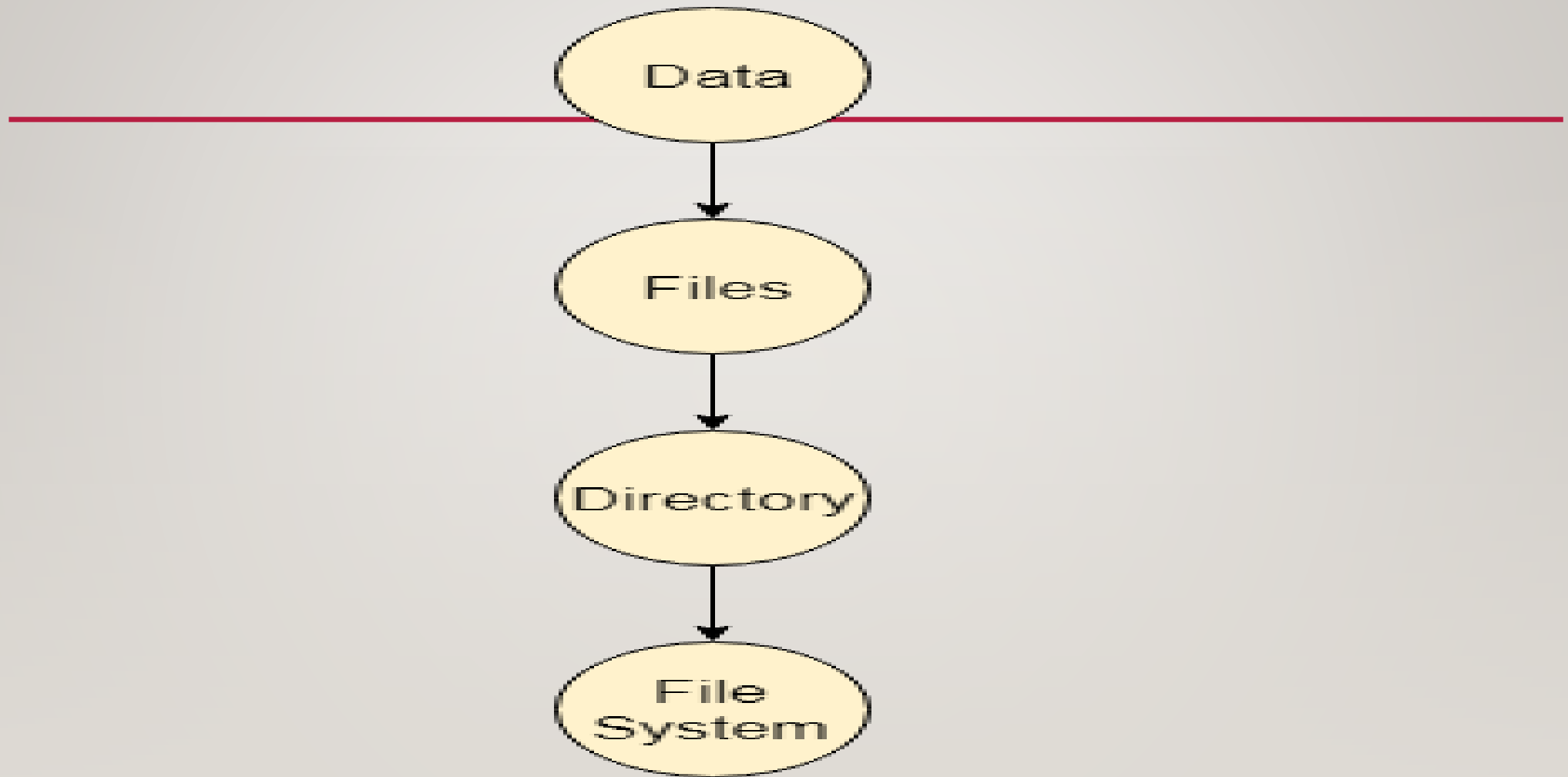
OPERATING SYSTEMS

M.M.A NISANSALA

DEPARTMENT OF COMPUTING

INTRODUCTION

- Storage Management in an Operating System (OS) is about how files and data are stored, organized, and retrieved efficiently on storage devices like hard disks, SSDs, optical disks, and flash drives.
- Since main memory (RAM) is volatile, data must be stored in secondary storage for permanent use.
- File System is the part of the OS that manages files, directories, and space allocation on secondary storage.



INTRODUCTION

What is a File?

- A file is a named collection of information stored on a storage device.
- Files may represent:
 - Programs (source code, executables).
 - Data (text, images, videos).
 - System info (configuration, logs).

INTRODUCTION

File Attributes (Metadata)

- Each file has associated information stored in a File Control Block (FCB/inode in UNIX)
 - Name – human-readable string.
 - Type – text, binary, executable, etc.
 - Location – address of file blocks on disk.
 - Size – current file length in bytes.
 - Protection – who can access and what rights (read/write/execute).
 - Timestamps – creation, last modified, last access.
 - Owner – user account

ATTRIBUTES OF THE FILE (METADATA)

- Each file has associated information stored in a File Control Block (FCB/inode in UNIX)

1. Name

Every file carries a name by which the file is recognized in the file system. One directory cannot have two files with the same name.

2. Identifier

Along with the name, Each File has its own extension which identifies the type of the file. For example, a text file has the extension **.txt**, A video file can have the extension **.mp4**.

3. Location

In the File System, there are several locations on which, the files can be stored. Each file carries its location as its attribute.

ATTRIBUTES OF THE FILE

4. Type

In a File System, the Files are classified in different types such as video files, audio files, text files, executable files, binary files and etc.

5. Size

The Size of the File is one of its most important attribute. By size of the file, we mean the number of bytes acquired by the file in the memory.

6. Protection

The Admin of the computer may want the different protections for the different files. Therefore each file carries its own set of permissions to the different group of Users.



ATTRIBUTES OF THE FILE

7. Timestamps

Every file carries a time stamp which contains the time and date on which the file is created , last modified and last accessed.

OPERATIONS ON THE FILE

- The various operations which can be implemented on a file such as read, write, open , delete , close etc. are called file operations.
- These operations are performed by the user by using the commands provided by the operating system.
- These operations are performed through system calls .

OPERATIONS ON THE FILE

1. Create Operation

This operation is used to create a file in the file system. Steps performed by OS are:

1. Allocate space in the directory
2. Create an empty File Control Block (FCB/ inode) with metadata
3. Assign default attributes like owner, permission, timestamps

Example: `fopen ("new. txt", "w")` in C.

OPERATIONS ON THE FILE

2. Open Operation

- When the user wants to open a file it prepare the file for access.
- When a file is opened:
 - OS loads file metadata into memory (open file table).
 - Returns a file descriptor (FD)/handle to the process.

Example: `fd = open("data.csv", O_RDONLY).`

OPERATIONS ON THE FILE

3. Read Operation

- Transfer data from the file (secondary storage) into memory.
 - OS checks permissions (Read access allowed?).
 - Moves the file pointer to the required position.
 - Transfers the requested number of bytes to the process buffer.

Example: `read(fd, buffer, 100)` → reads 100 bytes.

OPERATIONS ON THE FILE

4. Write operation

- Transfer data from process memory into the file (Secondary storage).
1. OS checks permissions (Write access allowed?).
 2. Writes data starting from the current file pointer location.
 3. Updates file size and modification timestamp.

Example: `write(fd, "hello", 5)` → writes 5 bytes.

OPERATIONS ON THE FILE

5. Closing operation

- Finish using a file and release resources.
- Steps are:
 - OS updates file metadata (timestamps, size).
 - Removes entry from open file table.
 - Frees the file descriptor.

Example: `close(fd)`.

OPERATIONS ON THE FILE

6. Delete Operation

- Remove a file permanently.
- Steps are :
 - Directory entry removed.
 - All associated disk blocks marked as free.
 - File Control Block (FCB) deallocated.

Example: `rm file.txt` in Linux.

FILE ATTRIBUTE OPERATIONS

- Besides content, OS also allows operations on metadata:
 - Change permissions (chmod in UNIX).
 - Change ownership (chown).
 - Change timestamps (touch).
 - Retrieve file info (stat, ls -l).

FILE STRUCTURE

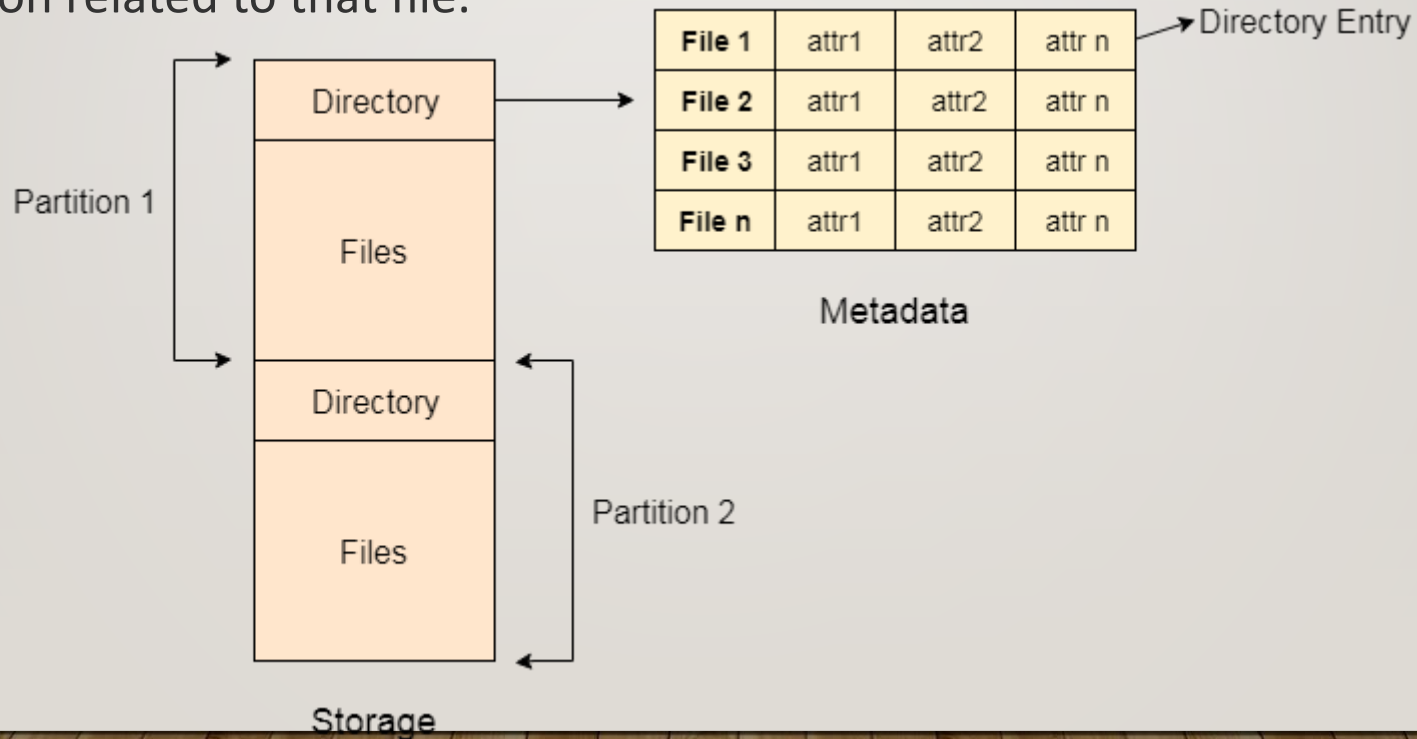
- How information is organized inside files:
 - **Byte sequence** (used in UNIX/Linux) – OS sees file as just a series of bytes.
 - **Record-based** – fixed-length or variable-length records (used in databases).
 - **Tree-structured files** – hierarchical indexing (e.g., XML, JSON).

DIRECTORY STRUCTURE

- Directory is a special file that keeps track of other files.
- It provides users with a way to organize files logically just like folders in a filing cabinet.
- Functions are:
 1. Organizing files into groups
 2. Allow navigate through hierarchy
 3. Maintain file metadata

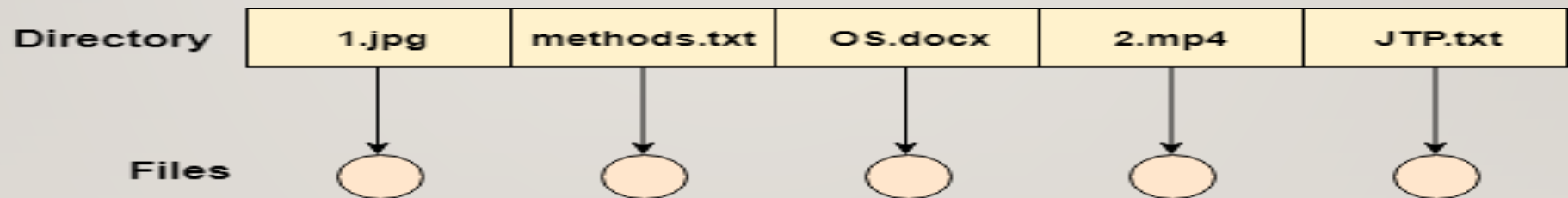
DIRECTORY STRUCTURES IN OS

- A directory entry is maintained for each file in the directory which stores all the information related to that file.



SINGLE LEVEL DIRECTORY

- The simplest method is to have one big list of all the files on the disk. The entire system will contain only one directory which is supposed to mention all the files present in the file system.
- The directory contains one entry per each file present on the file system.
- This is simple to implement and easy to understand. But name conflicts can happen as two users cannot have same name
- Examples: MS-DOS root directory in early system

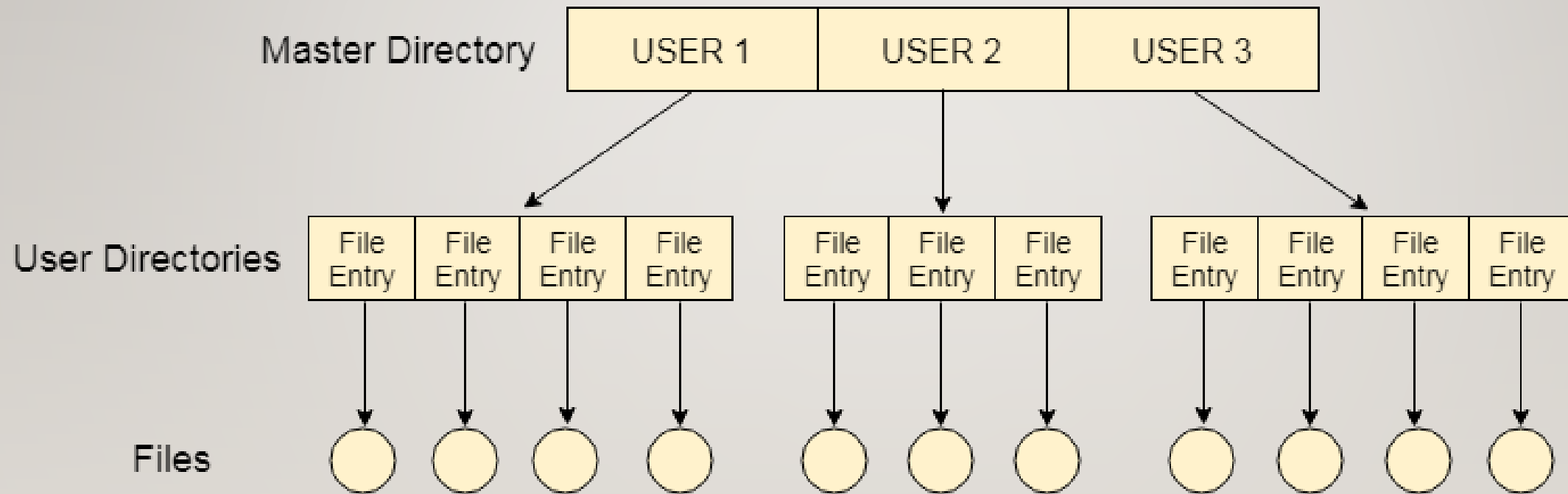


Single Level Directory

TWO LEVEL DIRECTORY

- In two level directory systems, we can create a separate directory for each user.
- There is one master directory which contains separate directories dedicated to each user.
- For each user, there is a different directory present at the second level, containing group of user's file.
- The system doesn't let a user to enter in the other user's directory without permission.
- Example: /home/user1/, /home/user2/ in Linux.

TWO LEVEL DIRECTORY



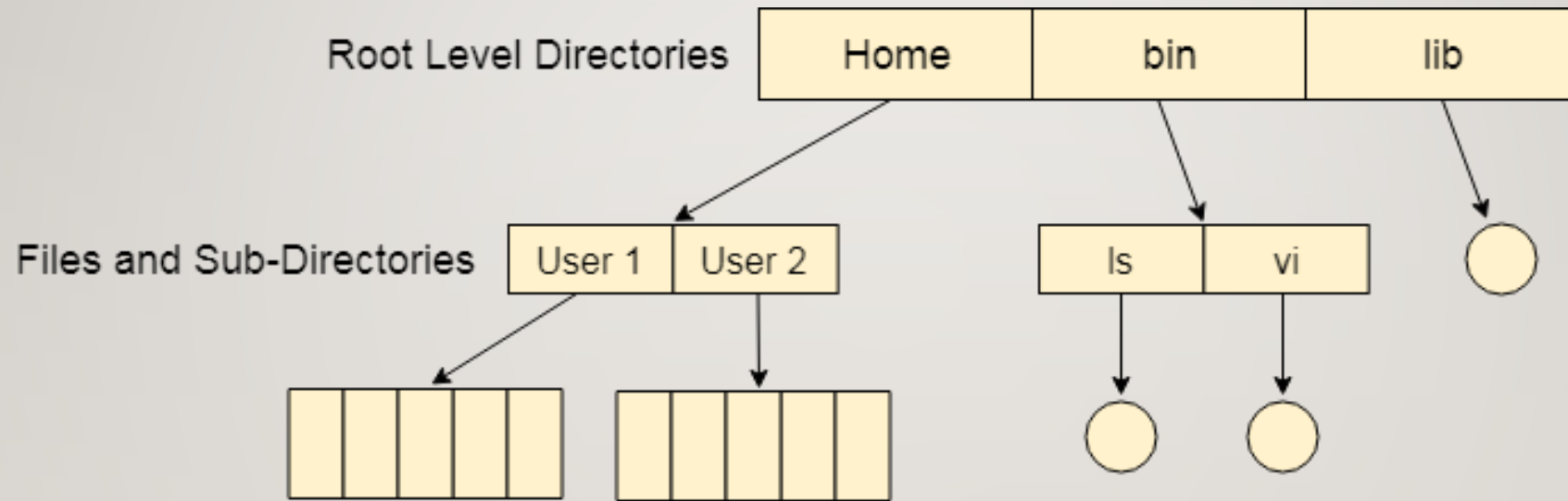
Two Level Directory

TREE-STRUCTURED DIRECTORY

- In Tree structured directory system, any directory entry can either be a file or sub directory. Directories organized as a hierarchical tree with subdirectories.
- This method is easy to use and support grouping into categories like documents, images and etc instead of keeping everything in a single place.
- This supports file sharing with absolute and relative paths.
- Disadvantage is searching deep in hierarchy may be slower.
- Example: Windows: C:\Users\Achini\Documents\OS\chapter9.docx

Linux: /home/achini/Documents/OS/chapter9.docx

TREE STRUCTURED DIRECTORY

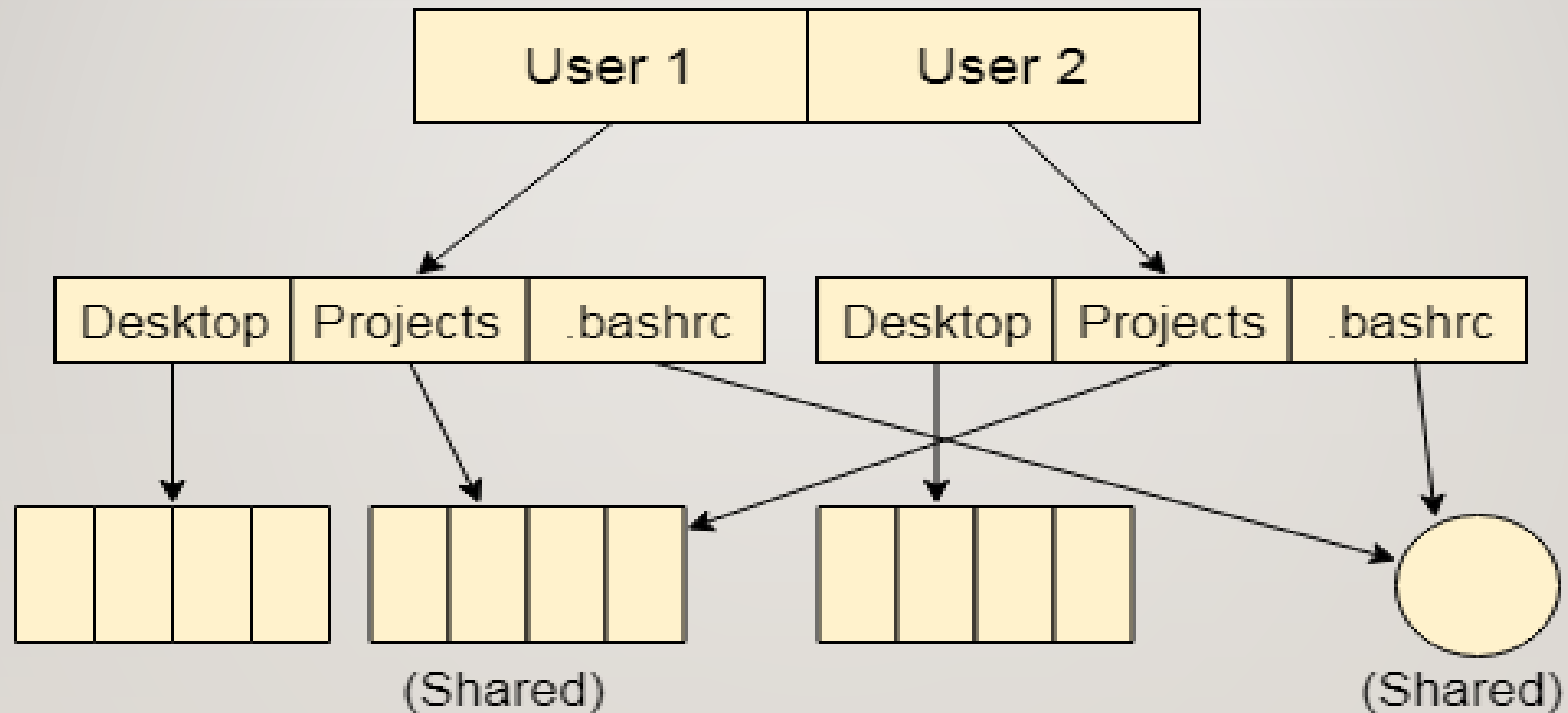


The Structured Directory System

ACYCLIC GRAPH-STRUCTURED DIRECTORY

- Similar to a tree structure, but files and directories can be shared by using links.
- It is a graph (not just a tree), but without cycles (no loop) that means you cannot start from one directory and keep following links to come back to the same directory.
- This saves space as no need to store multiple copies and it is keeps consistency.
- Example (Windows / Linux): You have a file report.docx in your Documents folder. You want to access the same file also from Projects folder. Instead of copying, you make a link/shortcut to the file.

ACYCLIC GRAPH-STRUCTURED DIRECTORY



Acyclic-Graph Structured Directory System

FILE SYSTEM MOUNTING

- A file system must be made accessible before use is called mounting.
- When mounted, it becomes part of the directory tree.
- Example: inserting a USB drive → OS mounts its file system → appears as E:\ or /media/usb.

FILE SYSTEM IMPLEMENTATION

- A disk is divided into logical parts:
 - Boot Control Block – contains code to boot the OS.
 - Volume Control Block (Superblock in Linux) – stores details like total size, free blocks, number of files.
 - Directory Structure – file names and pointers to File Control Blocks.
 - File Control Blocks (FCB/inode) – holds metadata of each file.

FILE ACCESS METHODS

I.Sequential Access

- In sequential access, the OS read the file word by word. A pointer is maintained which initially points to the base address of the file. If the user wants to read first word of the file then the pointer provides that word to the user and increases its value by 1 word. This process continues till the end of the file.
- Sequential access is mostly used due to the fact that most of the files such as text files, audio files, video files, etc need to be sequentially accessed.

FILE ACCESS METHODS

1 block

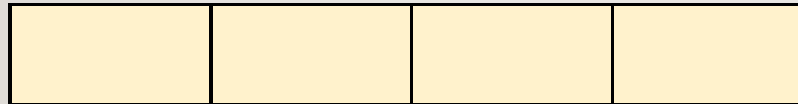
→ 512 Byte

10th

15th

11th

30th

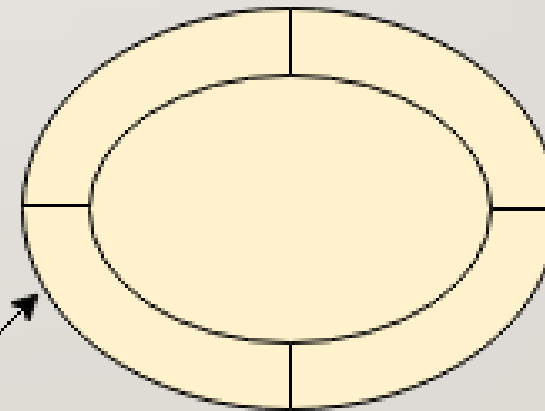


File

Sequential Access

1 Sector → 512 Byte

1 Sector



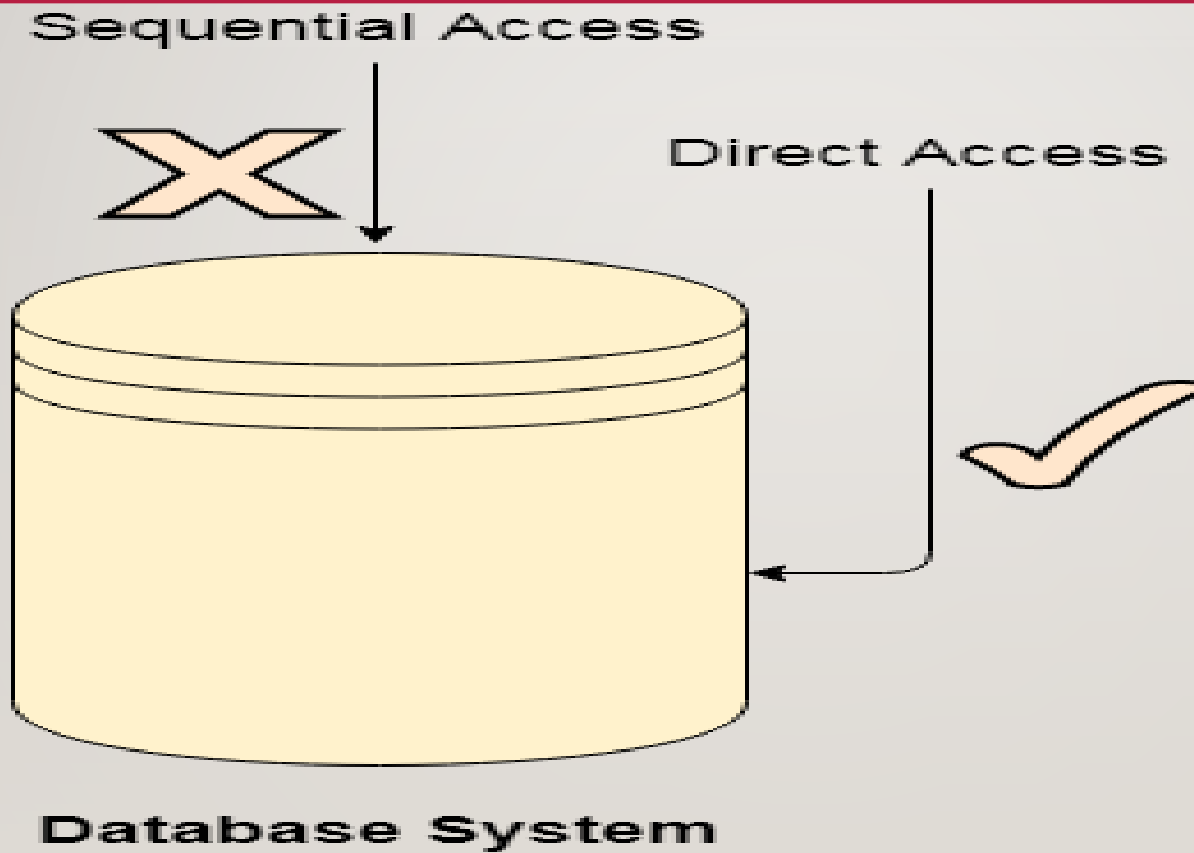
Hard Disk

FILE ACCESS METHODS

2. Direct access

- The Direct Access is mostly required in the case of database systems.
- In most of the cases, we need filtered information from the database. The sequential access can be very slow and inefficient in such cases.
- Suppose every block of the storage stores 4 records and we know that the record we needed is stored in 10th block.
- In that case, the sequential access will not be implemented because it will traverse all the blocks in order to access the needed record.

FILE ACCESS METHODS



FILE ACCESS METHODS

3. Indexed Access

- If a file can be sorted on any of the filed then an index can be assigned to a group of certain records.
- However, A particular record can be accessed by its index. The index is nothing but the address of a record in the file.
- In index accessing, searching in a large database became very quick and easy but we need to have some extra space in the memory to store the index value.

DISK SCHEDULING

- When many processes request disk access at the same time the OS decides the order request will be served. Goal is to minimize the seek time and improve throughput and response time.

Common Disk Scheduling Algorithms

I. First Come First Serve(First Come First Serve)

- Serve requests in the order they arrive. Fair and simple. But can cause long seek time.

DISK SCHEDULING

2. SSFT (Shortest Seek Time First)

- Choose the request closest to the current head position. Reduces average seek time. May cause starvation (far requests keep waiting).

3. SCAN (Elevator Algorithm)

- Disk arm moves in one direction, serving requests, then reverses. Fair, avoids starvation. More movement compared to SSTF.

4. C-SCAN (Circular SCAN)

- Like SCAN, but only moves in one direction. When the end is reached, jumps back to the beginning without serving. Provides uniform wait time.

FILE PROTECTION

- File protection ensures that only authorized users/processes can access files in allowed ways.

Types of File Access Rights

- Read (r) → view file contents.
 - Write (w) → modify or delete file.
 - Execute (x) → run file as a program.
 - Append (a) → add data at the end.
 - Delete (d) → remove file.
 - List (l) → show names in directory.
- 

PROTECTION MECHANISMS

1. Access Control Lists (ACLs)

- Each file has a list of users and their allowed rights

2. User Groups(Owner, Group, Others)

- This is used in Linux / Unix. Each file has a permissions for three categories as owner, group, others.

3. Password Protection

- File or director can be password protected.

4. Capability Lists(Ticketss)-

- Each user has a list of files and permissions.This is reverse of ACL.

EXAMPLES OF FILE SYSTEMS

- Different OS use different file system formats

1. FAT (File Allocation Table)

-Used in MS-DOS, USB drives, memory cards. Simple, but not efficient for large disks.Types are FAT12, FAT16, FAT32.

2. NTFS (New Technology File System)

-Default in Windows. Features: file compression, encryption, journaling (recovery after crash), large file support.

3. EXT (Extended File System)

-Common in Linux.Versions available are EXT2, EXT3, EXT4. EXT4 supports large disks, journaling, and faster performance.

EXAMPLES OF FILE SYSTEM

4. HFS+ / APFS (Apple File Systems)

-HFS+ used in older macOS. APFS (Apple File System) → optimized for SSDs, snapshots, encryption.

5. ISO 9660

-Used for CDs/DVDs.

6. exFAT

-Designed for flash drives and memory cards .Works in Cross-platform (works in Windows, macOS, Linux).

Thank You!